

4.7 OpenAPI and Swagger to Generate the Api Documentation

• Introduction :

- In modern software development, APIs serve as the **communication layer** between different systems, applications, and services. As the number of APIs grows, maintaining **accurate** and **up-to-date** documentation becomes increasingly important.
- **OpenAPI** and **Swagger** solve this problem by providing a **standardized way** to describe, document, visualize, and test REST APIs.

• What is OpenAPI ?

- **OpenAPI** is an industry-standard **specification** used to describe RESTful APIs.
- It acts as a **contract** between API producers and API consumers by defining:
 - Available endpoints
 - HTTP methods (GET, POST, PUT, DELETE, etc.)
 - Request parameters
 - Request body structure
 - Response body structure
 - HTTP status codes
 - Authentication and authorization requirements
- OpenAPI itself is **not** a tool or library. It is simply a **specification** that defines how API documentation should be structured.
- **Example:**

```
"paths": {  
  "/students/{id}": {  
    "get": {  
      "summary": "Get student by ID",  
      "responses": {  
        "200": {  
          "description": "Success"  
        }  
      }  
    }  
  }  
}
```

- The above example describes an API endpoint using the **OpenAPI specification**.

• What is Swagger ?

- **Swagger** is a collection of tools built around the OpenAPI specification.
- Swagger provides tools that can:
 - Generate API documentation
 - **Visualize** APIs through a web interface
 - Allow developers to **test** APIs directly from the browser
 - Generate client SDKs and server stubs
- The most commonly used Swagger tool is **Swagger UI**, which provides an interactive web page displaying all available APIs.

4.7 OpenAPI and Swagger to Generate the Api Documentation

• OpenAPI vs Swagger ?

- Many developers use these terms interchangeably, but they are not the same.

OpenAPI	Swagger
Specification	Toolset
Defines how APIs should be described	Implements the specification
Industry Standard	Collection of tools
API Contract	API Visualization and Testing
Ex. OpenAPI Document	Ex. Swagger UI, Swagger Editor, Swagger Codegen

- **Simple Analogy:**

- Java → Programming Language
- IntelliJ → Tool

Similarly:

- OpenAPI → Specification
- Swagger → Tool

• Why Do We Need OpenAPI vs Swagger ?

- **Without** API documentation:
 - Frontend developers must ask backend developers for endpoint details.
 - API consumers may misunderstand request and response formats.
 - Documentation becomes outdated as APIs evolve.
- **With** OpenAPI and Swagger:
 - Documentation is automatically generated.
 - Documentation stays synchronized with code.
 - APIs can be tested directly from a browser.
 - Frontend and backend teams can work independently.

• OpenAPI in Spring Boot :

- Spring Boot applications can automatically generate OpenAPI documentation by analyzing:
 - `@RestController`
 - `@GetMapping`
 - `@PostMapping`
 - `@PutMapping`
 - `@DeleteMapping`
 - `@RequestBody`
 - `@PathVariable`
 - DTO classes
- The generated specification is exposed through a REST endpoint.

4.7 OpenAPI and Swagger to Generate the Api Documentation

- **Example:**

```
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/{id}")
    public StudentResponseDto getStudentById( @PathVariable Long id) {
        return null;
    }
}
```

- Spring Boot automatically generates documentation for this endpoint.

- **Implementing OpenAPI and Swagger in Spring Boot :**

- **Add Dependency:**

```
<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.8.9</version>
</dependency>
```

- After adding the dependency reload the maven and start the application again, SpringDoc automatically scans all controllers and generates API documentation.

- **Access OpenAPI Specification :**

- Open: <http://localhost:8080/v3/api-docs>

- **Access Swagger UI :**

- Open: <http://localhost:8080/swagger-ui/index.html>

- **Customizing Documentation :**

- OpenAPI annotations can be used to enrich the generated documentation.
- **Api Documentation:** Use @Operation annotation to edit Api documentation.

```
@Operation(
    summary = "Get Student By ID",
    description = "Fetches a student using the provided student ID"
)
@RestController
@RequestMapping("/students")
public class StudentController {

    @GetMapping("/{id}")
    public StudentResponseDto getStudentById( @PathVariable Long id) {
        return null;
    }
}
```

- **DTO Documentation:** Use @Schema annotation to provide extra info about DTO in swagger ui.

```
@Data
@AllArgsConstructor
@NoArgsConstructor
public class StudentDTO {

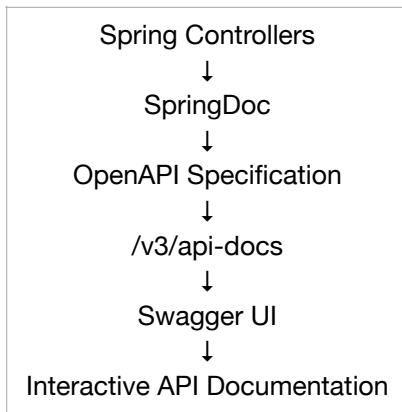
    private Long id;

    @Schema(description="name of a Student.")
    private String name;
}
```

This information appears automatically in Swagger UI.

4.7 OpenAPI and Swagger to Generate the Api Documentation

• Internal Working :



• Benefits of OpenAPI and Swagger :

- **Automatic Documentation:** Documentation updates automatically whenever the code changes.
- **Better Collaboration:** Frontend, backend, QA, and external consumers can work independently.
- **Interactive Testing:** APIs can be executed directly from Swagger UI without using Postman.
- **Reduced Maintenance:** No need to maintain separate Word documents, PDFs, or spreadsheets.
- **Standardization:** Provides a consistent industry-standard format for API contracts.

• Production Considerations :

- Swagger UI is often disabled in production environments for security reasons.
- Common practice: `springdoc.swagger-ui.enabled=false` in application.properties.
- However, the OpenAPI specification may still be generated internally and used by API gateways, client generators, and integration tools.

• Interview Answer :

- OpenAPI is an industry-standard specification used to describe REST APIs, including endpoints, request structures, response structures, parameters, authentication requirements, and status codes.
- Swagger is a collection of tools built around the OpenAPI specification that provides features such as interactive API documentation, testing interfaces, and code generation.
- In Spring Boot, SpringDoc automatically generates OpenAPI documentation from controller annotations and exposes it through the `/v3/api-docs` endpoint, while Swagger UI provides a browser-based interface for exploring and testing APIs through `/swagger-ui/index.html`.

All the required properties: <https://springdoc.org/#:-:text=5.1.%20springdoc%2Dopenapi%20core%20properties>